



UNIVERSIDAD NACIONAL
DE EDUCACIÓN A DISTANCIA

Escuela Técnica Superior de
Ingeniería Informática



UNIVERSIDAD
COMPLUTENSE DE MADRID

Facultad de
Informática

[TÍTULO DEL TRABAJO]

Martin Augusto Reigadas Teran

Director/a: [Director/a]

Co-director/a: [Co-director/a]

Trabajo de Fin de Máster

Máster Universitario
en Ingeniería de Control y Sistema

[Fecha de presentación/Convocatoria]

Agradecimientos

[Quisiera dedicar ...]

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Palabras clave: palabra1, palabra2, palabra3

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Keywords: keyword1, keyword2, keyword3

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Propuesta y objetivos	2
1.3. Estructura del documento	3
2. Fundamentos teóricos	5
2.1. Aprendizaje por Refuerzo	5
2.2. Aprendizaje por Refuerzo Profundo	9
2.2.1. Redes Neuronales Artificiales como Aproximadores de Funciones . . .	9
2.2.2. Taxonomía y Algoritmos Típicos de DRL	10
2.3. Cinemática Típica de un AGV	11
2.3.1. Modelo de Tracción Diferencial	12
2.3.2. Modelo de Dirección de Ackermann (Modelo de Bicicleta)	12
2.4. Cinemática Utilizada	13
3. Estado del Arte	15
3.1. Algoritmos de Aprendizaje por Refuerzo para Control Reactivo	15
3.2. Percepción Sensorial Multimodal y Observabilidad Parcial	16
3.3. Aprendizaje por Currículo y Navegación Segura	16
4. Diseño y arquitectura	19
4.1. Formulación del Problema como POMDP	19
4.2. Arquitectura de control	20
4.3. Modelo Cinemático del AGV sobre Rejilla	21
4.3.1. Modelo con Giro y Avance (GridRobot)	21
4.3.2. Modelo sin Giro y Traslación Directa (GridRobotNotTurning)	21
4.4. Espacio de Observaciones Perceptuales Locales	22
4.5. Función de Recompensa Paramétrica	23
4.6. Arquitecturas de Redes Neuronales DQN y PPO	23
4.6.1. Aproximación de Valor de Acción (DQN)	24
4.6.2. Actor-Crítico Asimétrico con PPO	25

4.6.3. Extensión Recurrente Propuesta (PPO-LSTM)	25
5. Configuración Experimental y Análisis de Resultados	27
5.1. Configuración de los Escenarios de la Rejilla	27
5.2. Hiperparámetros de Entrenamiento de los Algoritmos	27
5.3. Espacio de Acciones Discreto	27
5.4. Función de Recompensa Compuesta	28
5.5. Mecanismo de Reinicio Suave (Soft Reset)	29
5.6. Arquitecturas de Redes Neuronales DQN y PPO	29
5.7. Arquitectura	29
6. Resultados	31
6.1. Configuración de los Escenarios de la Rejilla	31
6.2. Hiperparámetros de Entrenamiento de los Algoritmos	31
6.3. Métricas de Evaluación de Navegación	32
6.4. Estructura de las Figuras y Curvas de Aprendizaje	32
7. Conclusiones y trabajos futuros	35
Bibliografía y referencias	36
A. Ejemplos en L^AT_EX	41
A.1. Referencias, citas y bibliografía	41
A.1.1. Referencias	41
A.1.2. Bibliografía	41
A.2. Figuras	42
A.3. Tablas	42
A.4. Código	43
A.5. Listas	44
A.6. Ecuaciones	44

Índice de figuras

2.1. Ciclo clásico de interacción Agente-Entorno en el marco del Aprendizaje por Refuerzo (RL).	6
2.2. Esquema cinemático clásico planar de un robot móvil (vista superior) para modelado físico continuo.	13
4.1. Arquitectura de control DRL	20
4.2. Diagrama visual de conexiones de la arquitectura de la red SimpleQNetwork de control adaptada a los espacios de acción discretos del AGV ($ A = 3$ para GridRobot y $ A = 4$ para GridRobotNotTurning).	26
5.1. Diagrama visual de conexiones de la arquitectura de la red SimpleQNetwork de control de avance y giro del AGV.	29
6.1. Curvas de convergencia del entrenamiento de los agentes DRL en el entorno de rejilla simple estática.	33
6.2. Comparación visual en 2D de las trayectorias resultantes del AGV autónomo reactivo evadiendo obstáculos de rejilla.	33
A.1. Ejemplo de figura	42

Índice de tablas

4.1. Topología y dimensiones de las redes neuronales implementadas para los agentes reactivos. La dimensión de entrada D_{obs} varía según la configuración del entorno (resolución del LiDAR y uso de acción previa).	24
5.1. Parámetros de entrenamiento del optimizador Adam y del buffer de experiencia.	28
5.2. Parámetros y dimensiones estructurales de las redes neuronales DQN y PPO implementadas.	30
6.1. Parámetros de entrenamiento del optimizador Adam y del buffer de experiencia.	31
6.2. Esqueleto comparativo de rendimiento de navegación y tasa de éxito (promedio sobre 200 episodios de evaluación).	32
6.3. Esqueleto comparativo de consumo energético del AGV y latencia del control reactivo en tiempo real.	32
A.1. Ejemplo de tabla	43

Capítulo 1

Introducción

1.1. Motivación

La automatización de los procesos intralogísticos es uno de los pilares de la transición hacia la manufactura flexible de la Industria 4.0 [Hu et al., 2020]. En este contexto, los Vehículos de Guiado Automático (AGVs, por sus siglas en inglés) desempeñan un papel fundamental en el transporte autónomo de componentes dentro de plantas industriales y almacenes [Xue et al., 2022a]. Tradicionalmente, la navegación de AGVs se ha fundamentado en planificadores de rutas clásicos que requieren información cartográfica exhaustiva previa (navegación basada en mapas, utilizando SLAM o guías físicas) [Sivashangaran and Eskandarian, 2023a]. Sin embargo, estas metodologías tradicionales conllevan elevados costes de instalación, mantenimiento y actualización del mapa cuando el entorno sufre alteraciones estructurales frecuentes, lo cual es habitual en entornos logísticos modernos de manufactura esbelta [Xue et al., 2022b].

Los métodos clásicos de planificación local de rutas en tiempo real, tales como los Campos Potenciales Artificiales (APF, por sus siglas en inglés), presentan la ventaja de poseer una estructura matemática simple y un bajo coste computacional, idóneos para controles reactivos rápidos [Pérez et al., 2022]. No obstante, sufren de limitaciones teóricas insalvables, destacando la alta propensión de quedar atrapados en situaciones de mínimos locales estables, donde la fuerza de atracción hacia el objetivo se cancela exactamente con la fuerza de repulsión de los obstáculos cercanos [Pérez et al., 2022]. Por otro lado, las heurísticas globales de búsqueda basadas en muestreo estocástico como RRT o en rejillas como A^* resultan computacionalmente costosas de recalcularse en entornos dinámicos en tiempo real y son sensibles al ruido sensorial local [Oh et al., 2025].

Para solventar estas barreras tecnológicas, surge la navegación reactiva libre de mapas (*map-less navigation*) basada en el Aprendizaje por Refuerzo Profundo (DRL, por sus siglas en inglés) [Guo et al., 2025a]. Mediante DRL, es posible entrenar redes neuronales profundas que sirvan como políticas de control reactivo extremo a extremo (*end-to-end*), mapeando

directamente lecturas de sensores de rango locales (como nubes de puntos de LiDAR) y coordenadas relativas de la meta a comandos de actuación discretos o continuos [Sivashangaran et al., 2023]. Este paradigma elimina por completo la dependencia de mapas globales previos y posicionamiento SLAM robusto, lo cual dota al AGV de una robustez y flexibilidad excepcionales ante escenarios industriales impredecibles y estáticos con obstáculos no mapeados [Xue et al., 2022b].

1.2. Propuesta y objetivos

El presente trabajo de investigación propone el diseño, implementación y evaluación de un sistema de navegación reactiva map-less para un AGV operando en un entorno de rejilla discreta bidimensional con obstáculos estáticos desconocidos. La política de navegación reactiva del AGV procesa información perceptual multimodal ligera: un vector de sensado de LiDAR submuestreado en secciones regulares locales, el vector de posición relativa a la meta expresado en coordenadas polares (rumbo y distancia) y el estado de velocidad previa del robot. A partir de estas observaciones locales, la política entrenada con DRL genera decisiones sobre comandos discretos de dirección (avanzar, girar a la izquierda, girar a la derecha) con el fin de guiar al AGV de manera segura hacia la meta.

Para superar el severo problema de la dispersión de recompensa (*sparse reward*) inherente a las tareas de navegación de largo horizonte temporal en entornos con obstáculos, se formula una función de recompensa compuesta con penalizaciones por proximidad que moldea el comportamiento seguro del robot. Asimismo, se incorpora un Handler de Interrupciones en tiempo real (*is_stuck*) con un buffer de memoria temporal que activa un mecanismo de reinicio suave (*soft reset*) de la exploración estocástica cuando el AGV cae en una zona de bloqueo o mínimo local estático de la rejilla.

Objetivo General: Diseñar y evaluar una arquitectura de control de navegación reactiva libre de mapas (*map-less*) para un robot móvil tipo AGV en entornos discretos de rejilla con obstáculos estáticos utilizando algoritmos de Aprendizaje Profundo por Refuerzo (DRL).

Objetivos Específicos:

1. Formular matemáticamente el problema de navegación reactiva como un Proceso de Decisión de Markov Parcialmente Observable (POMDP) aplicable a entornos con sensores locales y sin mapa previo [Md et al., 2024].
2. Diseñar un vector de observaciones multimodal que combine lecturas discretizadas de LiDAR en secciones angulares y rumbo relativo a la meta para controlar la explosión dimensional del estado.
3. Desarrollar una función de recompensa compuesta que integre penalizaciones continuas de proximidad y un mecanismo heurístico de desatasco mediante reinicios suaves de exploración (*soft reset*).

4. Implementar y comparar las arquitecturas de red neuronal de aproximación de valor y política (como DQN y PPO) utilizando frameworks modernos basados en PyTorch.

Acrónimo	Definición
AGV	Vehículo de Guiado Automático (Automated Guided Vehicle)
DRL	Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning)
DQN	Red Q Profunda (Deep Q-Network)
PPO	Optimización de Política Próxima (Proximal Policy Optimization)
POMDP	Proceso de Decisión de Markov Parcialmente Observable

5. Validar el rendimiento de la política aprendida en diversos escenarios estáticos simulados de rejilla frente a los enfoques clásicos de Campos Potenciales Artificiales [Pérez et al., 2022].

1.3. Estructura del documento

La presente memoria de tesis de maestría se estructura de la siguiente manera:

- El **Capítulo 1 (Introducción)** contextualiza la motivación, los objetivos específicos y la propuesta del proyecto.
- El **Capítulo 2 (Estado del Arte)** revisa de manera exhaustiva la literatura científica existente sobre el uso de DRL en robótica móvil, comparando algoritmos de control, percepción multimodal y el aprendizaje curricular.
- El **Capítulo 3 (Diseño y arquitectura)** detalla formalmente el modelo matemático del POMDP, la arquitectura de control, la cinemática discreta del robot de rejilla, la estructura del LiDAR por sectores angulares, la función de recompensa y las arquitecturas de las redes DQN/PPO aplicadas.
- El **Capítulo 4 (Configuración Experimental y Análisis de Resultados)** introduce el esqueleto del escenario de evaluación, los hiperparámetros de entrenamiento de las redes y las métricas de rendimiento sugeridas para su futura cumplimentación de datos reales.
- El **Capítulo 5 (Conclusiones y Trabajo Futuro)** resume los aportes científicos fundamentales y propone líneas de investigación futuras.

Capítulo 2

Fundamentos teóricos

En este capítulo se presentan los fundamentos teóricos necesarios para comprender el desarrollo metodológico y la experimentación de este trabajo de fin de máster. En primer lugar, se expone el marco matemático del Aprendizaje por Refuerzo (RL) clásico, introduciendo la formulación de los Procesos de Decisión de Markov (MDP) y sus ecuaciones fundamentales de Bellman. Posteriormente, se detalla el Aprendizaje por Refuerzo Profundo (DRL) como una extensión para resolver espacios de alta dimensionalidad mediante redes neuronales artificiales, describiendo sus arquitecturas y algoritmos más destacados en la literatura. A continuación, se describen los modelos cinemáticos planos tradicionales utilizados típicamente para vehículos de guiado automático (AGV), tales como la tracción diferencial y el modelo de bicicleta de Ackermann. Finalmente, se justifica formalmente el modelo de cinemática simplificada discreta paso a paso implementado en este trabajo, detallando cómo la eliminación del tiempo de integración continua optimiza de manera drástica la velocidad de convergencia y la estabilidad del entrenamiento de los agentes reactivos.

2.1. Aprendizaje por Refuerzo

El Aprendizaje por Refuerzo (RL, por sus siglas en inglés, *Reinforcement Learning*) constituye un área de la inteligencia artificial y del aprendizaje automático enfocado en cómo un agente autónomo puede aprender a tomar decisiones secuenciales óptimas interactuando dinámicamente con un entorno estocástico mediante un proceso de ensayo y error [Shakya et al., 2023a]. A diferencia de otros paradigmas de aprendizaje, en RL el agente no es guiado por un conjunto de datos etiquetados de antemano; en su lugar, aprende a asociar sus acciones con recompensas o penalizaciones escalares proporcionadas por el entorno como retroalimentación inmediata, con el fin de reforzar los comportamientos deseables y desincentivar los perjudiciales [Murphy, 2025]. Este ciclo interactivo continuo se ilustra de manera simplificada en la Figura 2.1, donde se muestra la retroalimentación clásica entre las acciones del agente y las respuestas de estado y recompensa del entorno.

Figura 2.1: Ciclo clásico de interacción Agente-Entorno en el marco del Aprendizaje por Refuerzo (RL).

Matemáticamente, la interacción entre el agente y el entorno en pasos discretos de tiempo $t = 0, 1, 2, \dots$ se formaliza rigurosamente bajo el marco de los Procesos de Decisión de Markov (MDP) [Murphy, 2025]. Un MDP se define formalmente como una tupla de cinco elementos $\langle S, A, P, R, \gamma \rangle$, donde [Shakya et al., 2023a, Murphy, 2025]:

- S representa el espacio o conjunto de todos los estados posibles del entorno [Murphy, 2025].
- A denota el espacio de acciones disponibles para el agente [Murphy, 2025].
- $P(s_{t+1} \mid s_t, a_t)$ es la función de probabilidad de transición estocástica del estado del sistema, que define la probabilidad de transicionar al estado s_{t+1} dado que el agente ejecutó la acción a_t estando en el estado s_t [Shakya et al., 2023a].
- $R(s_t, a_t, s_{t+1})$ representa la función de recompensa escalar de retroalimentación provista al agente tras realizar una transición [Shakya et al., 2023a, Murphy, 2025].
- $\gamma \in [0, 1)$ es el factor de descuento temporal que pondera la ganancia futura, determinando el grado de miopía o visión a largo plazo del agente [Murphy, 2025].

Para un MDP finito, la dinámica conjunta de transición y recompensa del entorno se define mediante la función de distribución de probabilidad conjunta, expresada formalmente como [Shakya et al., 2023a]:

$$p(s', r \mid s, a) \doteq \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a) \quad (2.1)$$

A partir de esta distribución conjunta, se puede deducir analíticamente la recompensa esperada del paso siguiente para cualquier par estado-acción como [Shakya et al., 2023a]:

$$r_{t+1}(s, a) \doteq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{r \in \mathbb{R}} r \sum_{s' \in S} p(s', r \mid s, a) \quad (2.2)$$

Asimismo, la ganancia o retorno acumulado con descuento futuro a partir de cualquier paso de tiempo t , denotado por G_t , se define como la suma ponderada de la secuencia de recompensas futuras escalares obtenidas por el agente [Shakya et al., 2023a]:

$$G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.3)$$

El retorno G_t cumple una propiedad recursiva fundamental para el desarrollo teórico y la formulación recursiva de los algoritmos de RL, la cual asocia el retorno en el instante actual

con el retorno en el instante subsecuente mediante la ecuación [Shakya et al., 2023a]:

$$G_t = R_{t+1} + \gamma G_{t+1} \quad (2.4)$$

El comportamiento del agente se rige por una política $\pi(a_t | s_t)$, la cual define una distribución de probabilidad sobre el espacio de acciones dada una observación del estado actual del sistema [Murphy, 2025]. El objetivo primordial de cualquier algoritmo de aprendizaje por refuerzo es descubrir una política óptima π^* que maximice la ganancia acumulada con descuento futuro o retorno acumulado esperado, definido formalmente como [Shakya et al., 2023a]:

$$J(\pi) = \mathbb{E}_\pi[G_0] = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] \quad (2.5)$$

Para evaluar la calidad de una política, los algoritmos estiman funciones de valor que miden cuán beneficioso es para el agente estar en un estado particular o ejecutar una acción en dicho estado. La función de valor de estado $V^\pi(s)$ representa el retorno esperado partiendo desde el estado s y siguiendo la política π en los pasos subsecuentes [Shakya et al., 2023a]:

$$V^\pi(s) = \mathbb{E}_\pi [G_t | s_t = s] \quad (2.6)$$

Por su parte, la función de valor de acción $Q^\pi(s, a)$ representa el retorno acumulado esperado tras ejecutar la acción a estando en el estado s , y seguir posteriormente la política π para la toma de decisiones [Shakya et al., 2023a]:

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t | s_t = s, a_t = a] \quad (2.7)$$

Las funciones de valor cumplen una propiedad recursiva fundamental conocida como la Ecuación de Expectación de Bellman, la cual descompone el valor de un estado como la suma de la recompensa inmediata esperada y el valor descontado de los estados sucesores [Murphy, 2025]:

$$V^\pi(s) = \sum_{a \in A} \pi(a | s) \sum_{s' \in S} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')] \quad (2.8)$$

$$Q^\pi(s, a) = \sum_{s' \in S} P(s' | s, a) \left[R(s, a, s') + \gamma \sum_{a' \in A} \pi(a' | s') Q^\pi(s', a') \right] \quad (2.9)$$

De igual forma, las funciones de valor bajo una política óptima satisfacen las Ecuaciones de Optimalidad de Bellman, que introducen el operador de maximización rígida sobre el

espacio de acciones [Shakya et al., 2023a]:

$$V^*(s) = \max_{a \in A} \left\{ \sum_{s' \in S} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')] \right\} \quad (2.10)$$

$$Q^*(s, a) = \sum_{s' \in S} P(s' | s, a) \left[R(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right] \quad (2.11)$$

Dentro de la literatura de RL, los métodos para aproximar la política óptima se dividen bajo tres taxonomías principales [Shakya et al., 2023a, Murphy, 2025]:

- **Basados en Modelo (Model-based) vs. Libres de Modelo (Model-free):** Los métodos basados en modelo asumen o aprenden una estimación de las probabilidades de transición P y recompensas R del entorno para realizar planeación [Shakya et al., 2023a]. En cambio, los métodos libres de modelo no requieren conocer la física o dinámica del entorno, aprendiendo de manera directa a través de las muestras de experiencia recolectadas por ensayo y error [Shakya et al., 2023a].
- **En-Política (On-policy) vs. Fuera-de-Política (Off-policy):** Los métodos on-policy evalúan e incrementan de manera asíncrona la misma política utilizada para recolectar datos en la simulación [Murphy, 2025]. Por el contrario, los esquemas off-policy evalúan y optimizan una política objetivo independiente del comportamiento o la política exploratoria empleada para muestrear las transiciones, permitiendo reutilizar datos históricos de transiciones previas [Shakya et al., 2023a].
- **Basados en Valor (Value-based) vs. Basados en Gradiente de Política (Policy-based):** Los esquemas basados en valor se enfocan en aproximar la función Q óptima para derivar implícitamente una política de toma de decisiones codiciosa [Murphy, 2025]. Los enfoques basados en gradiente de política optimizan de forma directa los parámetros de una red de política $\pi_\theta(a | s)$ mediante gradiente de política para maximizar los retornos acumulados de la ruta sin requerir tablas de valor explícitas [Shakya et al., 2023a].

Cuando los MDPs presentan espacios de estados continuos o de dimensiones gigantescas, la representación tabular convencional de valores se vuelve inviable debido a limitaciones de memoria computacional, un problema conocido como la maldición de la dimensionalidad [Shakya et al., 2023a]. Para resolver este cuello de botella técnico, surge el **Aprendizaje por Refuerzo Profundo (DRL)**, que fusiona el marco de decisión estocástico de RL con el poder de extracción de características jerárquicas de las redes neuronales profundas.

2.2. Aprendizaje por Refuerzo Profundo

El Aprendizaje por Refuerzo Profundo (DRL, por sus siglas en inglés, *Deep Reinforcement Learning*) revoluciona el campo del control y la navegación autónoma al incorporar Redes Neuronales Profundas (DNN) como aproximadores de funciones no lineales de alta fidelidad para representar políticas, funciones de valor o modelos de transición [Shakya et al., 2023a]. Al prescindir de la selección manual de características (*hand-crafted features*) de baja dimensionalidad, las DNNs permiten al agente procesar directamente observaciones crudas y multimodales de alta dimensionalidad (tales como lecturas de LiDAR o imágenes de cámaras frontales) y mapearlas de extremo a extremo a decisiones óptimas de control [Hu et al., 2020].

2.2.1. Redes Neuronales Artificiales como Aproximadores de Funciones

Una red neuronal artificial es una estructura jerárquica de procesamiento matemático que consta de múltiples capas interconectadas de neuronas o unidades de cómputo [Shakya et al., 2023a]. El tipo de red más representativo y utilizado como bloque base en DRL es el Perceptrón Multicapa o Red Totalmente Conectada (MLP, por sus siglas en inglés, *Multi-Layer Perceptron*) [Ge et al., 2024a]. Un MLP típico se compone de una capa de entrada que recibe las observaciones del entorno, una o más capas ocultas intermedias responsables de la extracción progresiva de características, y una capa de salida que emite las predicciones o distribuciones de probabilidad de acciones [Hu et al., 2020].

Cada neurona de una capa oculta ejecuta una suma ponderada de sus entradas, le asocia un parámetro constante de sesgo (*bias*) e introduce no linealidad mediante una función de activación no lineal [Shakya et al., 2023a]:

$$h_j = \sigma \left(\sum_i w_{ji} x_i + b_j \right) \quad (2.12)$$

Donde x_i son las señales de entrada, w_{ji} representa el peso sinóptico de la conexión, b_j es el sesgo y $\sigma(\cdot)$ es la función de activación. Las funciones de activación más comunes en el campo del DRL para introducir no linealidad en el proceso de optimización son [Shakya et al., 2023a]:

- **ReLU (Rectified Linear Unit):** Definida como $\sigma(x) = \max(0, x)$, es la más utilizada en capas ocultas intermedias debido a su simplicidad computacional y su capacidad para mitigar el desvanecimiento de gradientes.
- **Tanh (Tangente Hiperbólica):** Escala las salidas al intervalo $[-1, 1]$, siendo útil cuando las variables del estado o los rangos de las acciones requieren valores simétricos

bidireccionales alrededor de cero.

- **Sigmoid:** Comprime las señales al de rango $[0, 1]$, empleada típicamente en distribuciones probabilísticas o compresión de factores de ganancia.

2.2.2. Taxonomía y Algoritmos Típicos de DRL

En función de qué componente sea aproximado por la red neuronal profunda, los algoritmos de DRL se dividen de manera clásica en dos grandes categorías, junto con arquitecturas híbridas avanzadas [Shakya et al., 2023a]:

- **Algoritmos Basados en Valor (Value-based Methods):** Aproximan la función de valor de acción óptima $Q^*(s, a)$ mediante una red neuronal parametrizada por pesos θ , denotada como $Q_\theta(s, a)$ [Shakya et al., 2023a].
 - *DQN (Deep Q-Network):* Incorpora dos mecanismos cruciales de estabilidad para romper la correlación temporal de las muestras y estabilizar el entrenamiento: el buffer de replay de experiencia (Experience Replay Buffer) y una red Q objetivo (Target Network) congelada temporalmente para estabilizar los objetivos de actualización de Bellman [Hu et al., 2020].
 - *Double DQN (DDQN):* Desacopla la selección de la mejor acción de su respectiva evaluación utilizando dos conjuntos distintos de pesos de red, mitigando con éxito el sesgo de sobreestimación característico del DQN estándar [Shakya et al., 2023a].
 - *Dueling DQN:* Modifica la arquitectura de la red para de manera separada estimar la función de valor de estado $V(s)$ y la ventaja de acción $A(s, a)$, mejorando la precisión en estados donde muchas acciones tienen similar valor esperado [Shakya et al., 2023a].
- **Algoritmos Basados en Política y Actor-Crítico (Actor-Critic Methods):** Optimizan de forma directa la política mediante gradientes y resuelven el problema de la alta varianza mediante el uso coordinado de dos aproximadores de red [Shakya et al., 2023a]. El **Actor** representa la política $\pi_\theta(a | s)$ y se encarga de seleccionar acciones, mientras que el **Crítico** aproxima la función de valor $V_\phi(s)$ o $Q_\phi(s, a)$ y evalúa las decisiones tomadas por el actor [Sivashangaran and Eskandarian, 2023a].
 - *DDPG (Deep Deterministic Policy Gradient):* Algoritmo off-policy que adapta el concepto de DQN al control de espacios de acciones continuos y de alta dimensionalidad, emitiendo comandos de actuación deterministas [Sivashangaran and Eskandarian, 2023a].

- *TD3 (Twin Delayed DDPG)*: Extiende DDPG introduciendo críticos gemelos minimizados para combatir la sobreestimación del valor Q, actualizando retardadamente la red del actor y suavizando las acciones objetivo con ruido de exploración regularizado [Sivashangaran and Eskandarian, 2023a].
- *SAC (Soft Actor-Critic)*: Algoritmo off-policy fundamentado en el Aprendizaje por Refuerzo de Máxima Entropía [Xue et al., 2022a]. SAC optimiza de manera coordinada el retorno acumulado esperado y la entropía de la política, lo que incentiva de manera implícita una exploración sumamente agresiva e impide que la política colapse prematuramente en mínimos locales subóptimas [Xue et al., 2022a, Sivashangaran and Eskandarian, 2023a].
- *PPO (Proximal Policy Optimization)*: Algoritmo on-policy que destaca por su extraordinaria estabilidad matemática [Ge et al., 2024a]. PPO implementa una función de pérdida sustituta recortada (*clipped surrogate objective*) que limita de forma estricta la magnitud de los cambios en la actualización de la política entre iteraciones sucesivas, evitando colapsos catastróficos de aprendizaje sin requerir la complejidad computacional de las restricciones de segundo orden [Ge et al., 2024a].

2.3. Cinemática Típica de un AGV

El diseño y despliegue de sistemas de navegación autónoma reactiva exige comprender el comportamiento mecánico y cinemático de los Vehículos de Guiado Automático (AGV) que operan en entornos industriales y almacenes logísticos estructurados [Ye et al., 2023]. En el plano bidimensional continuo XY , la pose de un robot móvil se define comúnmente mediante el vector tridimensional de estado $\mathbf{x} = [x, y, \psi]^T$, donde (x, y) representan las coordenadas cartesianas del centro de masa, y ψ es el ángulo de rumbo u orientación angular con respecto al marco de referencia de la inercia global [Sivashangaran and Eskandarian, 2023a].

En la robótica terrestre, las dos configuraciones cinemáticas continuas más representativas y estudiadas en la literatura científica son:

2.3.1. Modelo de Tracción Diferencial

Este modelo cinemático se aplica a AGVs planos que constan de dos ruedas motrices coaxiales independientes controladas por motores individuales de velocidad, junto con ruedas locas pasivas (*casters*) delanteras y traseras para conservar el balance mecánico de la plataforma [Pérez et al., 2022]. La cinemática del robot diferencial continuo se define mediante un modelo no holonómico de rodadura pura sin deslizamiento lateral, cuyas ecuaciones diferenciales de movimiento responden a la velocidad lineal instantánea longitudinal $v(t)$ y a la

velocidad angular del chasis $\omega(t)$ [P'erez et al., 2022]:

$$\dot{x}(t) = v(t) \cos(\psi(t)) \quad (2.13)$$

$$\dot{y}(t) = v(t) \sin(\psi(t)) \quad (2.14)$$

$$\dot{\psi}(t) = \omega(t) \quad (2.15)$$

Sujeto a la restricción no holonómica de no deslizamiento lateral:

$$\dot{x}(t) \sin(\psi(t)) - \dot{y}(t) \cos(\psi(t)) = 0 \quad (2.16)$$

2.3.2. Modelo de Dirección de Ackermann (Modelo de Bicicleta)

El modelo de Ackermann es típico de vehículos industriales guiados o automóviles autónomos que orientan mecánicamente su eje directriz delantero [Sivashangaran and Eskandarian, 2023a]. Para simplificar el análisis y optimizar la velocidad de cómputo en el controlador del simulador, la cinemática se modela rigurosamente utilizando el modelo de bicicleta (*kinematic bicycle model*), agrupando las ruedas de cada eje en un solo punto equivalente situado en el centro de los ejes delantero y trasero [Sivashangaran and Eskandarian, 2023a]. Las ecuaciones cinemáticas de actualización se definen por:

$$\dot{x}(t) = v(t) \cos(\psi(t) + \beta(t)) \quad (2.17)$$

$$\dot{y}(t) = v(t) \sin(\psi(t) + \beta(t)) \quad (2.18)$$

$$\dot{\psi}(t) = \frac{v(t)}{l_r} \sin(\beta(t)) \quad (2.19)$$

Con el ángulo de deslizamiento del centro de masa $\beta(t)$ definido por:

$$\beta(t) = \tan^{-1} \left(\frac{l_r}{l_f + l_r} \tan(\delta(t)) \right) \quad (2.20)$$

Donde l_f y l_r representan las distancias desde el centro de masa del AGV hacia los ejes delantero y trasero respectivamente, $\delta(t)$ representa el ángulo de dirección de la rueda delantera de Ackermann, y $v(t)$ es la velocidad longitudinal del vehículo directriz [Sivashangaran and Eskandarian, 2023a].

2.4. Cinemática Utilizada

A diferencia de los modelos cinemáticos continuos tradicionales basados en ecuaciones diferenciales acopladas y tiempos de integración infinitesimales, la presente tesis de maestría adopta y fundamenta analíticamente una **cinemática simplificada y puramente discreta**

[Espacio reservado para el esquema cinemático diferencial y Ackermann del AGV]

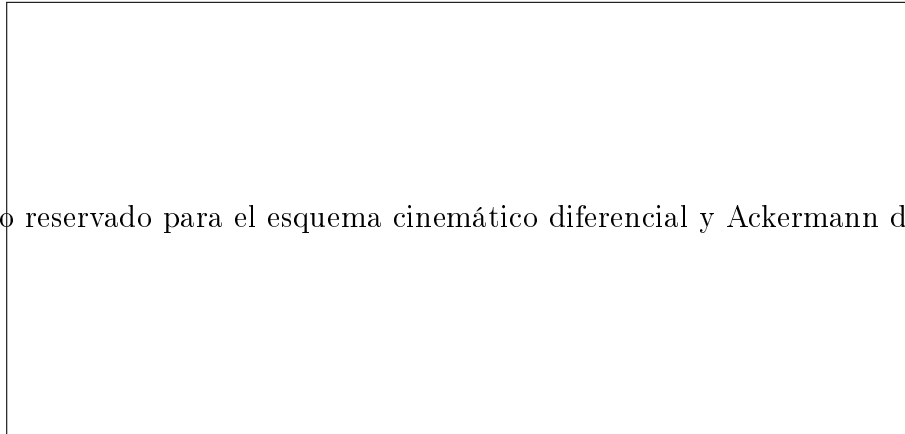


Figura 2.2: Esquema cinemático clásico planar de un robot móvil (vista superior) para modelado físico continuo.

de paso de iteración. En este modelo de simulación interactiva, el concepto clásico de velocidad física continua $v(t)$, aceleración lineal $a(t)$ o paso de tiempo infinitesimal diferencial Δt desaparece por completo [?]. El tiempo del sistema se modela estrictamente como una sucesión discreta de iteraciones o pasos de simulación del entorno de aprendizaje $t \rightarrow t + 1$.

La justificación teórica y práctica de este diseño metodológico simplificado responde a tres pilares esenciales:

1. **Aceleración Exponencial del Aprendizaje (Convergencia de DRL):** La simulación e integración de físicas continuas acopladas requiere de un cómputo masivo que ralentiza de forma severa el bucle de simulación, incrementando los tiempos de entrenamiento a varios días o semanas en CPU/GPU [Sivashangaran and Eskandarian, 2023a]. Al discretizar y unificar cada movimiento como una transición rígida unitaria (un paso de celda por iteración), el espacio de búsqueda del optimizador de DRL disminuye drásticamente, lo cual acelera la convergencia de las políticas reactivas en una fracción del tiempo requerido por métodos continuos tradicionales [Xue et al., 2022a].
2. **Estabilidad Absoluta y Evasión de Derivas Numéricas:** En escenarios industriales dinámicos con pasillos sumamente estrechos y curvas ortogonales, los modelos continuos con inercia rotacional y fuerzas de tracción reales son propensos a experimentar deslizamientos o derivas acumuladas debido al ruido de los motores. Estas imprecisiones físicas introducen inestabilidad en la actualización de las redes Q [Xue et al., 2022a]. El modelo discreto garantiza transiciones cinemáticas perfectas celda por celda sin inercia mecánica, facilitando al agente el aprendizaje geométrico de rutas colisión-free en rejillas regulares de ocupación (*GridMap*) [?].
3. **Consistencia y unificación con la Rejilla de Ocupación:** Al operar el AGV sobre un entorno discretizado por cuadrículas, la pose del robot se adapta naturalmente al espacio entero $\mathbb{Z}^2$. De esta manera, el procesamiento local de las distancias radiales

del sensor LiDAR (`GridLidar`) y la navegación relativa hacia el objetivo se evalúan de forma unificada paso a paso, eliminando la necesidad de fusionar mecánicas continuas de tracción diferencial con mallas de celdas estáticas discretas [?].

Este marco teórico de cinemática discreta paso a paso constituye la base para formular matemáticamente las transiciones específicas de movimiento de los robots implementados en este trabajo (`GridRobot` y `GridRobotNotTurning`), cuyas ecuaciones discretas de transición de pose y espacios de acciones unificados de cardinalidad $|A| = 3$ y $|A| = 4$ se desarrollarán rigurosamente en el Capítulo 4 (Metodología).

Capítulo 3

Estado del Arte

3.1. Algoritmos de Aprendizaje por Refuerzo para Control Reactivo

El desarrollo del DRL aplicado al guiado de robots autónomos se divide primordialmente entre algoritmos de optimización de gradiente de política directos *on-policy* y algoritmos basados en valor de acción de tipo crítico-actor *off-policy* [Pérez et al., 2022]. Entre los enfoques *on-policy*, el algoritmo de Optimización de Política Próxima (PPO, por sus siglas en inglés) destaca por su extraordinaria estabilidad numérica y robustez ante el ajuste de hiperparámetros [Shakya et al., 2023b]. PPO previene las actualizaciones de política excesivamente destructivas mediante una función de pérdida sustituta con restricciones por recorte del ratio de probabilidad de la acción, limitando la divergencia de Kullback-Leibler entre la política vieja y la nueva [Research, 2025]:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (3.1)$$

No obstante, dado que PPO es *on-policy*, padece de una baja eficiencia en el uso de muestras, requiriendo millones de interacciones directas con el entorno de simulación para converger hacia una política robusta [Oh et al., 2025].

Para solventar la ineficiencia de muestreo, se han extendido aproximaciones *off-policy* de crítico-actor continuas como DDPG y su variante de críticos retrasados gemelos TD3 [Sivashangaran and Eskandarian, 2023a]. Adicionalmente, el algoritmo Soft Actor-Critic (SAC) destaca como un método altamente eficiente basado en la maximización de la entropía máxima, el cual optimiza de manera conjunta el retorno esperado y el grado de aleatoriedad de la política [Sivashangaran and Eskandarian, 2023b]. Esta adición fomenta una exploración muy agresiva y previene que el actor converja prematuramente a mínimos locales estáticos

o cuellos de botella subóptimos en la navegación [Xue et al., 2022b]:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))] \quad (3.2)$$

En entornos logísticos de rejilla donde las acciones son intrínsecamente discretas (avanzar, girar), las redes Q profundas (DQN) de tipo *off-policy* y sus extensiones de doble estimación (Double DQN) resultan óptimas por su simplicidad de convergencia teórica y por permitir el reuso masivo de muestras previas del buffer de replay [Hu et al., 2020].

3.2. Percepción Sensorial Multimodal y Observabilidad Parcial

La navegación reactiva sin mapas se enmarca típicamente bajo la formulación matemática de un Proceso de Decisión de Markov Parcialmente Observable (POMDP), dado que el AGV solo dispone de lecturas de sensores de rango locales y carece de la posición global de los obstáculos en un plano cartesiano unificado [Md et al., 2024]. El sensado local con LiDARs discretos o continuos presenta problemas críticos de oclusión geométrica (puntos ciegos detrás de paredes) y ruidos sensoriales de rebote estocástico [Sivashangaran et al., 2023]. Para atenuar la dimensionalidad de las observaciones crudas de LiDAR (que pueden llegar a cientos de rayos densos), la literatura propone técnicas de discretización modular o compresión mediante autoencoders geométricos robustos (GDAE) [Guo et al., 2025b].

Por otra parte, la fusión de información sensorial visual profunda con datos de rango LiDAR se consolida mediante el uso de redes convolucionales preentrenadas (como MobileNetV3) en esquemas autoasociativos estáticos para codificar el entorno visual en un vector de baja dimensionalidad espacial [Research, 2025, Xue et al., 2022b]. No obstante, la pérdida de observabilidad temporal en el POMDP requiere el uso de memorias recurrentes profundas, tales como celdas LSTM o arquitecturas autorregresivas basadas en Transformers de decisión (Decision Transformers) [Ge et al., 2024a,b]. Estos componentes de memoria temporal permiten procesar una ventana histórica de observaciones y acciones previas $h_t = (o_1, a_1, \dots, o_t)$, extrayendo el “estado de creencia” estocástico implícito (*belief state*) indispensable para desambiguar estados geoméricamente idénticos y evadir de forma activa obstáculos dinámicos u oclusiones severas [Md et al., 2024, Oh et al., 2025].

3.3. Aprendizaje por Currículo y Navegación Segura

El entrenamiento de un AGV map-less desde cero bajo un esquema tradicional de DRL se enfrenta al cuello de botella de la “recompensa dispersa” (*sparse reward*) [Ye et al., 2023]. Si el espacio de estados es grande y la meta es de dimensiones reducidas en comparación con

la arena de pruebas, la probabilidad estocástica de que el agente alcance la meta mediante exploración aleatoria pura al inicio del entrenamiento es prácticamente nula, lo que provoca estancamientos en el gradiente de optimización [Oh et al., 2025]. Para mitigar esto, se ha introducido el Aprendizaje por Currículo Automático (ACL, por sus siglas en inglés) [Xue et al., 2022b].

La metodología propuesta por el algoritmo *NavACL* y su versión extendida con retroalimentación del valor de acción de los críticos, *NavACL-Q*, proponen dinámicamente escenarios de entrenamiento de dificultad adaptativa progresiva [Xue et al., 2022b]. *NavACL-Q* evalúa métricas geométricas de los mapas logísticos (distancia geodésica, holgura del agente y del objetivo ante obstáculos, y el ángulo de rumbo inicial) combinándolas con la estimación de valor Q predicha por la red crítica del agente, para proponer poses iniciales que se sitúen en la frontera del conocimiento actual de la política [Xue et al., 2022a]. A medida que el AGV incrementa su tasa de éxito local, el currículo de forma automática expande la distancia inicial de navegación y la complejidad de los laberintos. Adicionalmente, el Aprendizaje por Refuerzo Seguro (Safe RL) incorpora la teoría de Optimización de Política con Restricciones (CPO, por sus siglas en inglés), donde la función de costo modela explícitamente el riesgo de colisión catastrófica, asegurando que la política explore el espacio con garantías de seguridad física bajo restricciones de barreras de control [Gao et al., 2025a].

Capítulo 4

Diseño y arquitectura

En este capítulo se describe de manera detallada la metodología propuesta para resolver el problema de navegación autónoma reactiva de un Vehículo de Guiado Automático (AGV) en entornos de rejilla con obstáculos estáticos. En primer lugar, se formaliza el problema de navegación map-less bajo el marco teórico de los Procesos de Decisión de Markov Parcialmente Observables (POMDP). Posteriormente, se exponen los modelos cinemáticos discretos implementados (para los robots GridRobot y GridRobotNotTurning), describiendo las transiciones paso a paso por iteración, así como el procesamiento del sensor GridLidar por secciones para controlar la dimensionalidad del estado. Finalmente, se detalla la formulación matemática de la función de recompensa compuesta asimétrica y la estructura de las redes neuronales profundas (DQN y PPO) utilizadas por los agentes.

4.1. Formulación del Problema como POMDP

Dado que el AGV no tiene acceso global al plano de los obstáculos estáticos de la rejilla, el proceso se modela rigurosamente como un Proceso de Decisión de Markov Parcialmente Observable (POMDP), definido por la tupla matemática $\langle S, A, T, R, \Omega, O, \gamma \rangle$ [Md et al., 2024], donde:

- S es el espacio de estados del sistema que incluye la pose real absoluta del AGV y la configuración de los obstáculos de la rejilla.
- A es el espacio discreto de acciones ejecutables por el robot de guiado automático, el cual varía de acuerdo con el modelo cinemático implementado.
- $T(s_{t+1}|s_t, a_t)$ es la función estocástica de transición de estados de la rejilla.
- $R(s_t, a_t)$ es la recompensa asimétrica compuesta que guía y penaliza al agente.
- Ω representa el espacio de observaciones locales perceptuales del robot.

- $O(o_t|s_t, a_{t-1})$ denota la función de distribución probabilística de la observación sensorial local.
- $\gamma \in [0, 1)$ es el factor de descuento temporal que pondera la ganancia futura.

Ante la falta de observabilidad completa del estado geométrico s_t , el agente estima inductivamente un estado de creencia o vector de confianza $b_t(s) \in \mathcal{B}$ sobre la distribución del estado real mediante actualización recursiva bayesiana [Md et al., 2024]:

$$b_{t+1}(s') = \eta O(o_{t+1}|s', a_t) \sum_{s \in S} T(s'|s, a_t) b_t(s) \quad (4.1)$$

donde η es una constante normalizadora de probabilidad [Md et al., 2024].

4.2. Arquitectura de control

La arquitectura de control se basa en un entorno matricial en el que el robot se desplaza o rota secuencialmente por iteración. Este entorno envía el vector de observación (o_t), que describe el estado del robot y su entorno inmediato, al agente DRL para la selección de la acción discreta (a_t).

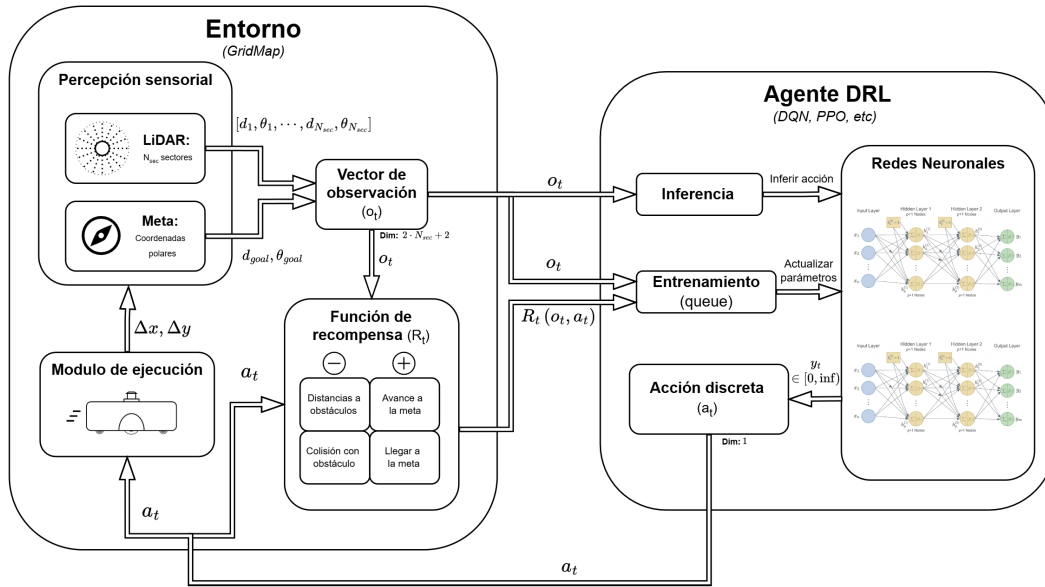


Figura 4.1: Diagrama de la arquitectura de control DRL y flujo de información entre el entorno y el agente.

Como se observa en la figura 4.1, las observaciones (o_t) y la señal de recompensa ($R_t(o_t, a_t)$) se almacenan en una cola de experiencia para el posterior entrenamiento de las redes neuronales del agente.

4.3. Modelo Cinemático del AGV sobre Rejilla

El vehículo de guiado automático (AGV) opera sobre una cuadrícula discreta de celda fija $\Delta_{\text{cell}} = 10$ unidades de simulación. Y se formalizan dos modelos cinemáticos alternativos en espacio discreto.

4.3.1. Modelo con Giro y Avance (GridRobot)

El estado del AGV en el instante t se define por el vector de pose $\mathbf{x}_t = [x_t, y_t, \theta_t]^T \in \mathcal{X}$, con $\mathcal{X} = \mathbb{Z}^2 \times \Theta$ y $\Theta = \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$. La transición de estado dada la acción $a_t \in \{0, 1, 2\}$ se rige por la ecuación diferencial discreta:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, a_t) \quad (4.2)$$

donde el operador de transición $\mathbf{f}(\mathbf{x}_t, a_t)$ se define vectorialmente como:

$$\mathbf{f}(\mathbf{x}_t, a_t) = \begin{bmatrix} \delta(a_t) \mathbf{d}(\theta_t) \\ \Delta\theta(a_t) \end{bmatrix} \quad (4.3)$$

Siendo $\mathbf{d}(\theta_t) = [\cos \theta_t, \sin \theta_t]^T \in \{-1, 0, 1\}^2$ el vector unitario de dirección cardinal, $\delta(a_t) = \mathbb{I}(a_t = 0)$ la función indicadora de traslación, y $\Delta\theta(a_t)$ el incremento angular modelado como:

$$\Delta\theta(a_t) = \begin{cases} 0^\circ & \text{si } a_t = 0 \text{ (Avanzar)} \\ +90^\circ \quad (\text{mód } 360^\circ) & \text{si } a_t = 1 \text{ (Giro Izquierda)} \\ -90^\circ \quad (\text{mód } 360^\circ) & \text{si } a_t = 2 \text{ (Giro Derecha)} \end{cases} \quad (4.4)$$

4.3.2. Modelo sin Giro y Traslación Directa (GridRobotNotTurning)

Para este modelo puramente holonómico discreto, la orientación se asume constante, reduciendo el espacio de estados a la posición cartesiana $\mathbf{p}_t = [x_t, y_t]^T \in \mathbb{Z}^2$. La ecuación de movimiento adopta la forma matricial compacta:

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{M} \mathbf{e}_{a_t} \quad (4.5)$$

donde $\mathbf{e}_{a_t} \in \mathbb{R}^4$ es el vector canónico unitario de la acción $a_t \in \{0, 1, 2, 3\}$, y $\mathbf{M} \in \mathbb{Z}^{2 \times 4}$ es la matriz de desplazamiento cardinal:

$$\mathbf{M} = \begin{bmatrix} 0 & 0 & 1 & -1 \\ -1 & 1 & 0 & 0 \end{bmatrix} \quad \text{correspondiente a } \{\text{Norte, Sur, Este, Oeste}\} \quad (4.6)$$

4.4. Espacio de Observaciones Perceptuales Locales

El estado observable por el agente en cada instante t , denotado como $\mathbf{o}_t \in \mathbb{R}^{D_{\text{obs}}}$, se estructura como un vector denso formado por la concatenación de la información exteroceptiva del sensor LiDAR y la odometría relativa hacia la meta. Inicialmente, este vector se define como:

$$\mathbf{o}_t = \begin{bmatrix} \mathbf{r}_{\text{LiDAR}} \\ \boldsymbol{\theta}_{\text{LiDAR}} \\ d_{\text{goal}} \\ \theta_{\text{goal}} \end{bmatrix} \in \mathbb{R}^{2N_{\text{sec}}+2} \quad (4.7)$$

donde $\mathbf{r}_{\text{LiDAR}}, \boldsymbol{\theta}_{\text{LiDAR}} \in \mathbb{R}^{N_{\text{sec}}}$ representan los vectores de distancias mínimas por sector y sus ángulos de incidencia asociados, respectivamente. Por su parte, d_{goal} y θ_{goal} indican la distancia cartesiana euclídea y el ángulo relativo desde el AGV hasta la meta. La lectura de distancia no normalizada para cada sector k se obtiene mediante:

$$r_{\text{LiDAR}}^{(k)} = \min_{\phi \in \text{Sector}_k} \{d_{\text{ray}}(\phi)\}, \quad k \in \{1, \dots, N_{\text{sec}}\} \quad (4.8)$$

Para abordar la pérdida de la propiedad de Markov inherente a la observabilidad parcial del entorno (POMDP), el espacio de observaciones se amplía incluyendo la memoria inercial de la última acción ejecutada, a_{t-1} . Adicionalmente, con el objetivo de estabilizar el cálculo de gradientes y acelerar la convergencia de la red neuronal, las magnitudes espaciales se normalizan al rango $[0, 1]$. La formulación final de la ecuación (4.7) resulta en:

$$\mathbf{o}_t = \begin{bmatrix} \bar{\mathbf{r}}_{\text{LiDAR}} \\ \boldsymbol{\theta}_{\text{LiDAR}} \\ \bar{d}_{\text{goal}} \\ \theta_{\text{goal}} \\ a_{t-1} \end{bmatrix} \in \mathbb{R}^{2N_{\text{sec}}+3} \quad (4.9)$$

donde $\bar{\mathbf{r}}_{\text{LiDAR}}$ contiene las distancias sectoriales normalizadas respecto al rango máximo del sensor d_{max} , calculadas como:

$$\bar{r}_{\text{LiDAR}}^{(k)} = \min_{\phi \in \text{Sector}_k} \left\{ \frac{d_{\text{ray}}(\phi)}{d_{\text{max}}} \right\}, \quad k \in \{1, \dots, N_{\text{sec}}\} \quad (4.10)$$

De manera análoga, $\bar{d}_{\text{goal}}$ representa la distancia cartesiana a la meta normalizada mediante d_{diag} , correspondiente a la diagonal del mapa cartográfico (distancia máxima franqueable en la rejilla):

$$\bar{d}_{\text{goal}} = \frac{d_{\text{goal}}}{d_{\text{diag}}} \quad (4.11)$$

4.5. Función de Recompensa Paramétrica

La señal de recompensa $r_t = \mathcal{R}(s_t, a_t)$ se formula mediante una combinación lineal de incentivos dispersos (*sparse*) y densos. Para permitir la experimentación entre diferentes modelos cinemáticos y casuísticas, los pesos de cada componente están parametrizados [?]:

$$r_t = r_{\text{term}} + r_{\text{prog}} - r_{\text{step}} - r_{\text{prox}} \quad (4.12)$$

Los componentes de la función se definen de la siguiente manera:

- r_{term} asigna la recompensa de los estados terminales (absorbentes), cuyos valores exactos ($R_{\text{win}}, R_{\text{fail}}, R_{\text{timeout}}$) se ajustan empíricamente según la complejidad del escenario:

$$r_{\text{term}} = \begin{cases} R_{\text{win}} > 0, & \text{si el AGV alcanza la meta} \\ R_{\text{fail}} < 0, & \text{si colisiona con un obstáculo} \\ R_{\text{timeout}} < 0, & \text{si excede el horizonte máximo } T_{\text{max}} \\ 0, & \text{durante las transiciones regulares} \end{cases} \quad (4.13)$$

- $r_{\text{prog}} = w_d(d_{t-1} - d_t)$ es un término de *reward shaping* basado en el potencial. Escala con el peso w_d la reducción de la distancia cartesiana entre el AGV y la meta respecto a la iteración anterior.
- r_{step} es una pequeña penalización escalar aplicada en cada transición regular. Su objetivo es evitar oscilaciones innecesarias e incentivar políticas de ruta mínima [Guo et al., 2025b].
- r_{prox} introduce una penalización de seguridad repulsiva si el agente invade un margen de proximidad crítico. Dada la distancia mínima censada $d_{\text{min}} = \min(\mathbf{r}_{\text{LiDAR}})$ y un radio de seguridad parametrizable D_{safe} :

$$r_{\text{prox}} = \begin{cases} \lambda_{\text{prox}} \left(1 - \frac{d_{\text{min}}}{D_{\text{safe}}}\right), & \text{si } d_{\text{min}} < D_{\text{safe}} \\ 0, & \text{en caso contrario} \end{cases} \quad (4.14)$$

donde el factor λ_{prox} pondera el coste máximo impuesto antes del impacto inminente.

4.6. Arquitecturas de Redes Neuronales DQN y PPO

Las aproximaciones de las funciones de valor y las políticas del actor-crítico se modelan mediante perceptrones multicapa (MLP) totalmente conectados. La dimensión de la capa de entrada, D_{obs} , se adapta dinámicamente tanto a la resolución del sensor ($N_{\text{sec}} \in \{4, 8\}$) como a la configuración del estado perceptual (inclusión o exclusión de la memoria inercial a_{t-1}).

En consecuencia, el tamaño de la entrada abarca los posibles valores $D_{\text{obs}} \in \{10, 11, 18, 19\}$. Del mismo modo, la capa de salida se ajusta a la cardinalidad del espacio de acciones discreto $|\mathcal{A}| \in \{3, 4\}$, dependiente del modelo cinemático activo.

El diseño estructural paramétrico de cada modelo se detalla en el Cuadro 5.2.

Tabla 4.1: Topología y dimensiones de las redes neuronales implementadas para los agentes reactivos. La dimensión de entrada D_{obs} varía según la configuración del entorno (resolución del LiDAR y uso de acción previa).

Red Neuronal	Capa / Tipo	Dimensión (Nodos)	Activación
GridAgentNet (Agente Base)	Entrada	D_{obs}	Ninguna
	Oculto 1 (Lineal)	253	ReLU
	Oculto 2 (Lineal)	56	ReLU
	Salida (Lineal)	$ \mathcal{A} $	Ninguna (Q-Values)
SimpleQNetwork (Agente DQN)	Entrada	D_{obs}	Ninguna
	Oculto 1 (Lineal)	H	ReLU
	Oculto 2 (Lineal)	H	ReLU
	Salida (Lineal)	$ \mathcal{A} $	Ninguna (Q-Values)
PolicyNetwork (Actor PPO)	Entrada	D_{obs}	Ninguna
	Oculto 1 (Lineal)	$H = 128$	ReLU
	Oculto 2 (Lineal)	64	ReLU
	Salida (Lineal)	$ \mathcal{A} $	Softmax
ValueNetwork (Crítico PPO)	Entrada	D_{obs}	Ninguna
	Oculto 1 (Lineal)	$H = 128$	ReLU
	Oculto 2 (Lineal)	64	ReLU
	Salida (Lineal)	1	Ninguna ($V(s)$)

4.6.1. Aproximación de Valor de Acción (DQN)

La arquitectura del *Deep Q-Network* (DQN) se orienta a aproximar la función de valor de acción óptima $Q^*(s, a)$. A diferencia de los métodos de gradiente de política, la red genera predicciones escalares (Q-Values) para todas las acciones posibles de forma simultánea. En el marco de esta tesis, se han implementado dos variantes estructurales para la evaluación mediante Q-Learning:

- **Red Base Asimétrica (GridAgentNet):** Una topología estática diseñada como *baseline* inicial. Está compuesta por dos capas ocultas con un fuerte descenso dimensional (de 253 a 56 neuronas). Esta asimetría acentuada busca forzar una compresión rápida del espacio de características perceptuales antes de proyectarlo al espacio de acciones.
- **Red Q Paramétrica (SimpleQNetwork):** Una arquitectura modular y simétrica que permite ajustar dinámicamente tanto la profundidad (número de capas ocultas) como la

anchura (dimensión H) de la red. Favorece la ejecución de estudios de hiperparámetros sobre la capacidad de generalización del modelo, empleando un diseño estándar con capas lineales sucesivas de dimensión H .

En ambas variantes, la capa de salida carece de función de activación (*Linear*), lo que permite que los estimadores $Q(s, a)$ abarquen todo el rango de los números reales, un requisito indispensable para alcanzar los valores extremos de las recompensas dispersas (como $+100$ o -100). Durante la fase de inferencia, la selección de la acción óptima se realiza de manera determinista evaluando $a_t = \arg \max_a Q(\mathbf{o}_t, a)$.

4.6.2. Actor-Crítico Asimétrico con PPO

La arquitectura de optimización de política proximal (PPO) se divide en dos redes independientes sin pesos compartidos para evitar interferencias en el cálculo de los gradientes entre el objetivo de política y el de valor:

- **Actor (PolicyNetwork):** Actúa como la función de política estocástica $\pi_\theta(a_t | \mathbf{o}_t)$. La capa de salida aplica una función de activación *Softmax* sobre la dimensión de las acciones, generando una distribución de probabilidad categórica de la cual se muestrea la acción a ejecutar.
- **Crítico (ValueNetwork):** Estima la función de valor del estado $V_\phi(\mathbf{o}_t)$. Su salida es un único nodo escalar lineal, entrenado para minimizar el error cuadrático medio respecto a los retornos descontados observados empíricamente.

4.6.3. Extensión Recurrente Propuesta (PPO-LSTM)

Para abordar la pérdida de la propiedad de Markov derivada de las observaciones parciales de forma estructural, se propone una extensión de la topología reactiva introduciendo una arquitectura recurrente. Para garantizar la estabilidad de la optimización *on-policy*, se diseña utilizando una única capa LSTM. Arquitecturas con múltiples capas recurrentes incrementan la susceptibilidad al desvanecimiento del gradiente (*vanishing gradient*) sin ofrecer ventajas empíricas demostrables en entornos de navegación discreta espacial.

El flujo de procesamiento propuesto para la red recurrente se define como:

1. **Extractor de Características:** Una capa lineal inicial acoplada a una activación ReLU (e.g., 64 nodos) que procesa la observación cruda $\mathbf{o}_t \in \mathbb{R}^{D_{\text{obs}}}$.
2. **Núcleo Recurrente (Memoria):** Una capa LSTM con tamaño de estado oculto (h_t, c_t) de 64 unidades. Esta capa integra temporalmente las secuencias de características extraídas.

3. **Cabeceras (*Heads*):** La salida temporal de la LSTM alimenta directamente dos capas lineales finales separadas: una con activación Softmax para inferir la política $\pi(a_t \mid \mathbf{o}_{\leq t})$, y otra escalar para aproximar la función de valor $V(\mathbf{o}_{\leq t})$.

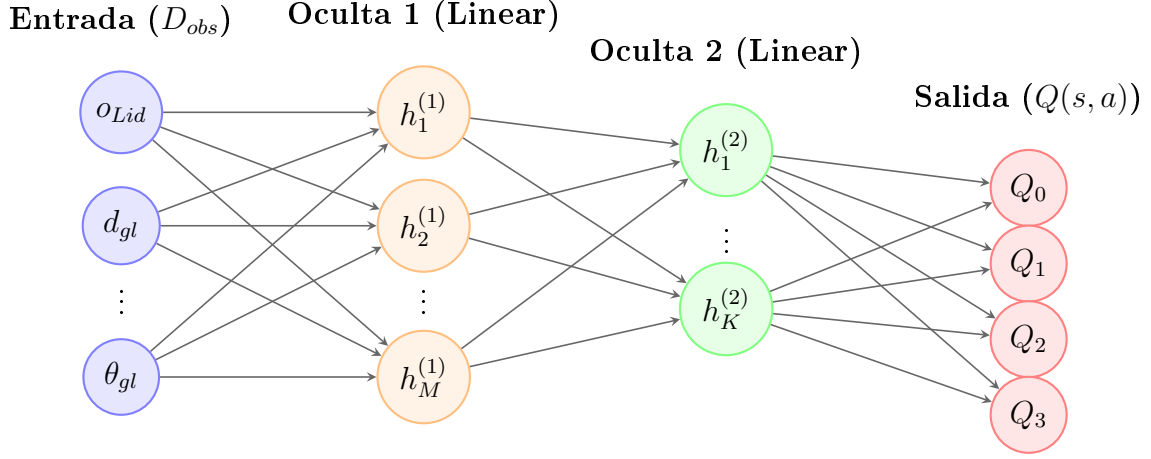


Figura 4.2: Diagrama visual de conexiones de la arquitectura de la red SimpleQNetwork de control adaptada a los espacios de acción discretos del AGV ($|A| = 3$ para `GridRobot` y $|A| = 4$ para `GridRobotNotTurning`).

Capítulo 5

Configuración Experimental y Análisis de Resultados

5.1. Configuración de los Escenarios de la Rejilla

Los experimentos numéricos y la evaluación de robustez se llevan a cabo sobre tres configuraciones de mapas estáticos cargados dinámicamente desde archivos de configuración serializados JSON [??]:

- `map_obstacle_free.json`: Escenario libre de obstáculos estáticos para validar la velocidad de convergencia teórica del AGV hacia la meta de navegación [?].
- `map_simple.json`: Escenario con un obstáculo geométrico rectangular central de rejilla fija para validar la maniobrabilidad básica de evasión de esquinas [?].
- `map_1_obstacle.json`: Escenario con un obstáculo asimétrico para validar el escape reactivo estocástico implementado [?].

5.2. Hiperparámetros de Entrenamiento de los Algoritmos

El entrenamiento de los agentes DQN y PPO se implementa utilizando los hiperparámetros estandarizados recopilados de tus scripts en el Cuadro 6.1 [??].

5.3. Espacio de Acciones Discreto

De acuerdo con el desarrollo lógico del software en el script `agent.txt`, el espacio de acciones se discretiza en tres decisiones de control de baja latencia para la navegación sobre la cuadrícula [?]:

Tabla 5.1: Parámetros de entrenamiento del optimizador Adam y del buffer de experiencia.

Hiperparámetro	Valor DQN / PPO	Descripción Técnica en el Código
Tasa de Aprendizaje (α_{lr})	10^{-3}	Tasa del optimizador Adam de la red de política
Factor de Descuento (γ)	0,99	Tasa de descuento de ganancias futuras [?]
Tamaño de Lote (<i>Batch size</i>)	64	Tamaño de muestra del optimizador estocástico
Pasos de Decaimiento (ϵ_{decay})	1,000,000	Pasos del decaimiento extendido de exploración
Capacidad del Buffer	100,000	Tamaño límite del buffer de replay de transición
Norma de Gradiente Máxima	5,0	Límite del recorte del gradiente (<i>gradient clipping</i>)

- **Acción 0 (Avanzar):** Comando para avanzar en línea recta a velocidad constante ($v = 1,0$ celda/paso, $\omega = 0,0$ rad/s).
- **Acción 1 (Girar Izquierda):** Comando de rotación pura a la izquierda ($v = 0,0$, $\omega = +\pi/2$ rad/s).
- **Acción 2 (Girar Derecha):** Comando de rotación pura a la derecha ($v = 0,0$, $\omega = -\pi/2$ rad/s).

5.4. Función de Recompensa Compuesta

La recompensa asimétrica compuesta $R(s_t, a_t)$ asocia incentivos dispersos de finalización y penalizaciones estocásticas continuas de seguridad para evitar colisiones [?]:

$$R(s_t, a_t) = R_{terminal} + \mathcal{R}_{progreso} - R_{time} - \mathcal{P}_{proximidad} \quad (5.1)$$

donde:

- $R_{terminal}$ es la recompensa estocástica de estados de parada terminales:

$$R_{terminal} = \begin{cases} +100,0, & \text{si AGV alcanza meta (éxito)} \\ -100,0, & \text{si colisiona con obstáculo} \\ -100,0, & \text{si excede límite temporal de pasos } T_{max} \end{cases} \quad (5.2)$$

- $\mathcal{R}_{progreso} = w_d \cdot (d_{t-1} - d_t)$ premia la aproximación radial real hacia la meta de destino.
- $R_{time} = 0,1$ es una penalización constante por paso de tiempo para forzar al AGV a tomar rutas directas y cortas [Guo et al., 2025b].
- $\mathcal{P}_{proximidad}$ es la penalización de seguridad activa si el AGV entra dentro de la zona

crítica definida por el radio de seguridad de la rejilla ($\mathcal{D}_{safety} = d_{max} \cdot \frac{2}{3}$) [?]:

$$\mathcal{P}_{proximidad} = \begin{cases} \mathcal{P}_{max} \cdot \left(1, 0 - \frac{d_{min}}{\mathcal{D}_{safety}}\right), & \text{si } d_{min} < \mathcal{D}_{safety} \\ 0, 0, & \text{en otro caso} \end{cases} \quad (5.3)$$

siguiendo los hiperparámetros del código real de entrenamiento, fijando la penalización máxima en $\mathcal{P}_{max} = 10^{-2}$ (0,01) [?].

5.5. Mecanismo de Reinicio Suave (Soft Reset)

Para prevenir los frecuentes atrapamientos y estancamientos de la política en zonas de pasillos estrechos o esquinas estáticas (mínimos locales de la rejilla), se define el manejador de interrupciones `InterruptHandler` [?]. El sistema monitorea continuamente una cola de doble extremo (`deque`) de tamaño fijo de las últimas acciones y recompensas acumuladas [?]. La condición de atasco se determina heurísticamente mediante la función `is_stuck`:

$$\text{is_stuck} = \text{std}(\mathbf{x}_{last_30}) < \epsilon_{threshold} \quad \wedge \quad a_t \in \{1, 2\} \quad (5.4)$$

Si el robot se encuentra oscilando o atascado, el manejador ejecuta de manera reactiva un reinicio suave del escenario de exploración (`do_soft_reset`) [?]. Este algoritmo purga el buffer de memoria temporal del replay e incrementa temporalmente el factor de exploración estocástica ϵ del agente a un valor alto ($\epsilon \leftarrow 0,8$) para forzar al AGV a escapar del obstáculo de la rejilla de forma reactiva sin interrumpir drásticamente la continuidad del proceso de aprendizaje [??].

5.6. Arquitecturas de Redes Neuronales DQN y PPO

Las aproximaciones de valor y políticas del actor-crítico se estiman mediante perceptrones multicapa totalmente conectados (MLP) cuyas capas y dimensiones se detallan en el Cuadro 5.2 y se visualizan estructuralmente en la Figura 5.1 [?].

Figura 5.1: Diagrama visual de conexiones de la arquitectura de la red SimpleQNetwork de control de avance y giro del AGV.

5.7. Arquitectura

Antes de empezar de ver los modelos,

Tabla 5.2: Parámetros y dimensiones estructurales de las redes neuronales DQN y PPO implementadas.

Red Neuronal	Capa / Tipo	Dimensión (Nodos)	Activación	Parámetro del Código
SimpleQNetwork (Agente DQN)	Entrada	$D_{obs} = 12$	Ninguna	Observaciones del sensor
	Oculto 1 (Linear)	253	ReLU	Extracción de características
	Oculto 2 (Linear)	56	ReLU	Homogeneización dimensional
	Salida (Linear)	3	Ninguna	Valores Q de acciones candidatas
ValueNetwork (Crítico PPO)	Entrada	$D_{obs} = 12$	Ninguna	Vector de estado percibido
	Oculto 1 (Linear)	$H = 64$	ReLU	Dimensión oculta del crítico
	Oculto 2 (Linear)	64	ReLU	Representación de valores
	Salida (Linear)	1	Ninguna	Estimación de valor $V(s)$

Capítulo 6

Resultados

6.1. Configuración de los Escenarios de la Rejilla

Los experimentos numéricos y la evaluación de robustez se llevan a cabo sobre tres configuraciones de mapas estáticos cargados dinámicamente desde archivos de configuración serializados JSON [??]:

- `map_obstacle_free.json`: Escenario libre de obstáculos estáticos para validar la velocidad de convergencia teórica del AGV hacia la meta de navegación [?].
- `map_simple.json`: Escenario con un obstáculo geométrico rectangular central de rejilla fija para validar la maniobrabilidad básica de evasión de esquinas [?].
- `map_1_obstacle.json`: Escenario con un obstáculo asimétrico para validar el escape reactivo estocástico implementado [?].

6.2. Hiperparámetros de Entrenamiento de los Algoritmos

El entrenamiento de los agentes DQN y PPO se implementa utilizando los hiperparámetros estandarizados recopilados de tus scripts en el Cuadro 6.1 [??].

Tabla 6.1: Parámetros de entrenamiento del optimizador Adam y del buffer de experiencia.

Hiperparámetro	Valor DQN / PPO	Descripción Técnica en el Código
Tasa de Aprendizaje (α_{lr})	10^{-3}	Tasa del optimizador Adam de la red de políticas
Factor de Descuento (γ)	0,99	Tasa de descuento de ganancias futuras [?]
Tamaño de Lote (<i>Batch size</i>)	64	Tamaño de muestra del optimizador estocástico
Pasos de Decaimiento (ϵ_{decay})	1,000,000	Pasos del decaimiento extendido de exploración
Capacidad del Buffer	100,000	Tamaño límite del buffer de replay de transición
Norma de Gradiente Máxima	5,0	Límite del recorte del gradiente (<i>gradient clip</i>)

6.3. Métricas de Evaluación de Navegación

Para la validación objetiva del rendimiento del AGV libre de mapas frente a baselines clásicos, se estructuran las siguientes métricas de evaluación en el Cuadro 6.2 y Cuadro 6.3 [Gao et al., 2025b].

Tabla 6.2: Esqueleto comparativo de rendimiento de navegación y tasa de éxito (promedio sobre 200 episodios de evaluación).

Algoritmo / Política	SR (%)	SRN (%)	CT (prom.)	SPL
Campos Potenciales Clásicos (APF) [P'erez et al., 2022]	0.00	0.00	0.00	0.00
Algoritmos Genéticos (AG-MLP) [P'erez et al., 2022]	0.00	0.00	0.00	0.00
DQN (GridAgentNet Básica)	0.00	0.00	0.00	0.00
DQN con Soft Reset (GridAgent_2_FIXED)	0.00	0.00	0.00	0.00
PPO Reactivo (Map-less)	0.00	0.00	0.00	0.00

Tabla 6.3: Esqueleto comparativo de consumo energético del AGV y latencia del control reactivo en tiempo real.

Algoritmo / Política	Pasos por Episodio	Consumo de Energía (E_{total})
Campos Potenciales Clásicos (APF)	—	—
Algoritmos Genéticos (AG-MLP)	0.00	—
DQN (GridAgentNet Básica)	0.00	0.00
DQN con Soft Reset (GridAgent_2_FIXED)	0.00	0.00
PPO Reactivo (Map-less)	0.00	0.00

6.4. Estructura de las Figuras y Curvas de Aprendizaje

Para el futuro vaciado de las curvas e imágenes del simulador de rejilla, se preparan las siguientes maquetaciones de figuras de LaTeX:

[Espacio reservado para las curvas de convergencia de recompensa acumulada vs pasos de entrenamiento de DQN, DQN con Soft Reset y PPO]

Figura 6.1: Curvas de convergencia del entrenamiento de los agentes DRL en el entorno de rejilla simple estática.

[Espacio reservado para las trayectorias espaciales 2D sobre la rejilla GridMap mostrando los puntos de colisión del APF y las trayectorias seguras logradas por DRL]

Figura 6.2: Comparación visual en 2D de las trayectorias resultantes del AGV autónomo reactivo evadiendo obstáculos de rejilla.

Capítulo 7

Conclusiones y trabajos futuros

Bibliografía

- Hao Hu, Xiaoliang Jia, Qixuan He, Shifeng Fu, and Kuo Liu. Deep reinforcement learning based agvs real-time scheduling with mixed rule for flexible shop floor in industry 4.0. *Computers & Industrial Engineering*, 149:106749, 2020. doi: 10.1016/j.cie.2020.106749.
- Honghu Xue, Benedikt Hein, Mohamed Bakr, Georg Schildbach, Bengt Abel, and Elmar Rueckert. Using deep reinforcement learning with automatic curriculum learning for mapless navigation in intralogistics. *Applied Sciences*, 12(6):3153, 2022a. doi: 10.3390/app12063153.
- Shathushan Sivashangaran and Azim Eskandarian. Deep reinforcement learning for autonomous ground vehicle exploration without a-priori maps. *Advances in Artificial Intelligence and Machine Learning*, 3(2):1198–1219, 2023a. arXiv preprint arXiv:2301.04036.
- Honghu Xue, Benedikt Hein, Mohamed Bakr, Georg Schildbach, Bengt Abel, and Elmar Rueckert. Using deep reinforcement learning with automatic curriculum learning for mapless navigation in intralogistics. MDPI Applied Sciences, 2022b.
- David Abad P'erez, Basil Mohammed Al-Hadithi, and Victor Cadix Mart'in. Navigation of mobile robots using neural networks and genetic algorithms. *IEEE Latin American Transactions*, 2022.
- David Abad P'erez, Basil Mohammed Al-Hadithi, and V'ictor Cadix Mart'in. Navigation of mobile robots using neural networks and genetic algorithms. *IEEE Latin American Transactions*, 2022. URL <https://latamt.ieeeer9.org/index.php/transactions/article/view/8506>.
- Junhyuk Oh, David Silver, et al. Discovering deep reinforcement learning rules. *Nature Research article_1743243135*, 2025.
- Dayu Guo, Yuan Zhu, and Ke Lu. Map-less navigation algorithm for autonomous vehicles based on deep reinforcement learning. *Advances in Computer, Signals and Systems*, 9(1): 123–130, 2025a. doi: 10.23977/acss.2025.090117.

- Shathushan Sivashangaran, Apoorva Khairnar, and Azim Eskandarian. Autovrl: A high fidelity autonomous ground vehicle simulator for sim-to-real deep reinforcement learning. *IFAC-PapersOnLine*, 56(3):475–480, 2023. doi: 10.1016/j.ifacol.2023.12.069.
- Abdul Quadir Md, Dibyanshu Jaiswal, Senthilkumar Mohan, Nisreen Innab, Riza Sulaiman, Mohammed Kbiri Alaoui, and Ali Ahmadian. A novel approach for self-driving car in partially observable environment using life long reinforcement learning. *Sustainable Energy, Grids and Networks*, 38:101356, 2024. doi: 10.1016/j.segan.2024.101356.
- Ashish Kumar Shakya, Gopinatha Pillai, and Sohom Chakrabarty. Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, 231:120495, 2023a. doi: 10.1016/j.eswa.2023.120495.
- Kevin P. Murphy. Reinforcement learning: An overview. Technical report, Technical Report / Monograph, December 2025.
- Lun Ge, Xiaoguang Zhou, Yongqiang Li, and Yongcong Wang. Deep reinforcement learning navigation via decision transformer in autonomous driving. *Frontiers in Neurorobotics*, 18:1338189, 2024a. doi: 10.3389/fnbot.2024.1338189.
- Xianfeng Ye, Zhiyun Deng, Yanjun Shi, and Weiming Shen. Toward energy-efficient routing of multiple agvs with multi-agent reinforcement learning. *Sensors*, 23(12):5615, 2023. doi: 10.3390/s23125615.
- Ashish Kumar Shakya, Gopinatha Pillai, and Sohom Chakrabarty. Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, 2023b.
- Action Space Reduction Research. Action space reduction strategies for reinforcement learning in autonomous driving. Preprint arXiv:2507.05251, 2025.
- Shathushan Sivashangaran and Azim Eskandarian. Deep reinforcement learning for autonomous ground vehicle exploration without a-priori maps. *Advances in Artificial Intelligence*, 2023b.
- Y. Guo, X. Zhu, and Y. Lu. Map-less navigation algorithm for autonomous vehicles based on deep reinforcement learning. *Advances in Computer, Signals and Systems*, 2025b.
- Lun Ge, Xiaoguang Zhou, Yongqiang Li, and Yongcong Wang. Deep reinforcement learning navigation via decision transformer in autonomous driving. *Frontiers in Neurorobotics*, 2024b.
- J. Gao, X. Pang, Q. Liu, and Y. Li. Hierarchical reinforcement learning for safe mapless navigation with congestion estimation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025a.

- Jianqi Gao, Xizheng Pang, Qi Liu, and Yanjie Li. Hierarchical reinforcement learning for safe mapless navigation with congestion estimation. ICRA 2025 Accepted Paper, 2025b.
- Danilo Dessi, Francesco Osborne, Diego Reforgiato Recupero, Davide Buscaldi, Enrico Motta, and Harld Sack. Ai-kg: an automatically generated knowledge graph of artificial intelligence, 2020.
- P. Chen, Y. Lu, V. W. Zheng, X. Chen, and B. Yang. Knowedu: A system to construct knowledge graph for education. *IEEE Access*, 6:31553–31563, 2018. doi: 10.1109/ACCESS.2018.2839607.
- Yuehua Qin, Han Cao, and Leyi Xue. Research and application of knowledge graph in teaching: Take the database course as an example. *Journal of Physics: Conference Series*, 1607:012127, aug 2020. doi: 10.1088/1742-6596/1607/1/012127. URL <https://doi.org/10.1088/1742-6596/1607/1/012127>.
- Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. A survey on knowledge graphs: Representation, acquisition and applications, 2020.

Apéndice A

Ejemplos en L^AT_EX

Este primer apéndice presenta ejemplos en L^AT_EX de cómo incluir referencias, citas bibliográficas, figuras, tablas o código. Este apéndice se deberá eliminar de la memoria antes de entregar el trabajo.

A.1. Referencias, citas y bibliografía

A.1.1. Referencias

Para poder referenciar un elemento dentro de la memoria hay que marcarlo con una etiqueta (`\label{[id]}`) que lo identifique inequívocamente. Es habitual utilizar identificadores representativos, por ejemplo, para marcar la introducción podemos utilizar una etiqueta como `\label{chap:introduccion}`

Una vez marcado el elemento (capítulo, sección, figura, tabla, ...) en L^AT_EX, utilizaremos el comando `ref` indicando a qué etiqueta queremos referenciar (`\ref{[id]}`). Por ejemplo, de esta manera podemos referenciar al capítulo de la introducción con 1. Podemos acompañarlo de un texto como "...como se vió en el capítulo 1..." o bien utilizar el comando `\autoref{[i]}`, que incluiría el tipo del elemento y aparecería en el texto como Capítulo 1, o incluso referenciarlo por el nombre del elemento con `\nameref{[id]}`, lo que se mostraría como Introducción.

A.1.2. Bibliografía

Para añadir una cita bibliográfica al documento tendremos que asegurarnos que en el fichero de bibliografía (bibliografía.bib) se encuentre la entrada bibliográfica correspondiente. Una vez que tengamos nuestra entrada bibliográfica utilizaremos el comando de cita de L^AT_EX indicando el identificador de la referencia bibliográfica, por ejemplo: `\cite{aikg}`, que se se visualizaría como Dessi et al. [2020].

Cuando queramos citar más de un trabajo en un mismo punto del documento se deberán

añadir a la cita separados por comas, por ejemplo: `\cite{chen_2018, Qin_2020}`, que se visualizaría como Chen et al. [2018], Qin et al. [2020]. En estos casos es habitual añadir las citas en orden cronológico de más antigua a más reciente.

Tal y como está configurada esta plantilla, por defecto el comando `\cite` realizará una cita textual (`\citet`), es decir, la cita forma parte del texto. Este tipo de citas se suelen realizar en frases como:

“Algunos ejemplos de grafos de conocimiento se presentan en Ji et al. [2020]”

Cuando la cita no forma parte del texto si no que se utiliza para reforzar una afirmación realizada en una frase, se debe utilizar una cita entre paréntesis (o corchetes). Para ello se usa el comando `\citep`, por ejemplo:

“Existe una gran variedad de aplicaciones de los grafos de conocimiento [Ji et al., 2020].”

A.2. Figuras

Para añadir una figura al documento utilizaremos el entorno `figure`, indicando la posición que debe tener dicha figura en el documento (h: here, t: top, b: botom; p: page), se recomienda en lo posible evitar de “!” que ignora todos los ajustes de los parámetros. El orden en el que se indiquen cada una de las opciones se tendrá en cuenta para colocar la figura, es decir si se indicase el orden [htbp] la figura primero se intentará colocar en el lugar que ocupa en el documento, si no se puede se intentará colocar al inicio (top) de la página, en caso que tampoco sea posible se intentará colocar al final (bottom) de la página y, por último en caso que no sea posible ninguna de las anteriores se colocará al inicio de una página nueva.



Figura A.1: Esta figura tiene una descripción al pie muy larga, por lo que añadiremos un título breve utilizando para ello los corchetes tras el comando `\caption`. La etiqueta de la figura (label) se incluirá al inicio del flotante de la figura para que cualquier referencia cruzada (ref) a la misma lleve al inicio del flotante.

A.3. Tablas

Para añadir una tabla se utilizará el entorno `table`, indicando al inicio de la tabla el título de la misma utilizando el `caption`. Un ejemplo puede verse en la Tabla A.1.

Tabla A.1: Esta tabla presenta un ejemplo con tres columnas y formato formal.

Model	Accuracy	Precision	Recall	F1-score	AUC
Modelo 1	0,33	0,75	0,72	0,42	0,21
Modelo 2	0,01	0,63	0,60	0,50	0,10
Modelo 3	0,03	0,93	0,33	0,04	0,42

A.4. Código

Para añadir código en la memoria utilizaremos el paquete `listing`, que permite mostrar código formateado en diversos lenguajes (Java, Python, C, ...).

```

1 import numpy as np
2
3 def incmatrix(genl1, genl2):
4     m = len(genl1)
5     n = len(genl2)
6     M = None #to become the incidence matrix
7     VT = np.zeros((n*m,1), int) #dummy variable
8
9     #compute the bitwise xor matrix
10    M1 = bitxormatrix(genl1)
11    M2 = np.triu(bitxormatrix(genl2),1)
12
13    for i in range(m-1):
14        for j in range(i+1, m):
15            [r,c] = np.where(M2 == M1[i,j])
16            for k in range(len(r)):
17                VT[(i)*n + r[k]] = 1;
18                VT[(i)*n + c[k]] = 1;
19                VT[(j)*n + r[k]] = 1;
20                VT[(j)*n + c[k]] = 1;
21
22            if M is None:
23                M = np.copy(VT)
24            else:
25                M = np.concatenate((M, VT), 1)
26
27            VT = np.zeros((n*m,1), int)
28
29    return M

```

A.5. Listas

Como en cualquier procesador de textos debemos diferenciar dos tipos de listas. Las listas no numeradas (o de viñetas) se definirán mediante entornos `itemize`.

- Elemento no numerado
- Elemento no numerado
- Elemento no numerado

Las listas numeradas se definirán mediante entornos `enumerate`. En ambos casos, cada elemento de la lista se genera utilizando el comando `item`.

1. Elemento 1

Con el comando `\` podemos insertar saltos de línea sin cambiar de párrafo.

2. Elemento 2. Con el comando `\textbf` podemos **enfaticar con negrita** un texto y con el comando `\textit` podemos *enfaticar con itálica* un texto.

3. Elemento 3

A.6. Ecuaciones

Cuando una ecuación va a ser referenciada desde el texto, en principio más de una vez, será necesario nombrar o numerar dicha ecuación (ver Ecuación A.1). Para añadir una ecuación numerada utilizaremos el entorno `equation`.

$$\begin{pmatrix} w_1, & w_2, & \dots, & w_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} + b = 0 \rightarrow \mathbf{w}^t \mathbf{x} + b = 0 \quad (\text{A.1})$$

Cuando la ecuación no va a ser referenciada desde el texto, podemos utilizar la versión de entorno no numerada `equation*`

$$\frac{1}{2} \|\mathbf{w}\| = \frac{1}{2} \sqrt{\sum w_i^2}$$

Es también posible añadir ecuaciones en línea con el texto, para ello se debe incluir la expresión matemática en L^AT_EX encerrada entre símbolos de dólar. Por ejemplo, la ecuación anterior se podría representar también en línea: $\frac{1}{2} \|\mathbf{w}\| = \frac{1}{2} \sqrt{\sum w_i^2}$.